אלגוריתמים – מבחן מסכם

פרופ' תמי תמיר

3.3.2024 – מועד א'

- משך המבחן: 3 שעות
- חומר עזר מותר: דפי העזר המצורפים + דף אחד שהביא/ה הסטודנט/ית.
- מותר לצטט ולהסתמך (מבלי להוכיח) על משפטים וטענות שהוכחו בכיתה או בתרגילי הבית.
- בכל שאלה או סעיף שמופיע לידם סימן ♣, ניתן לכתוב 'לא יודע/ת' ולקבל רבע מהניקוד (ערך תחתון).

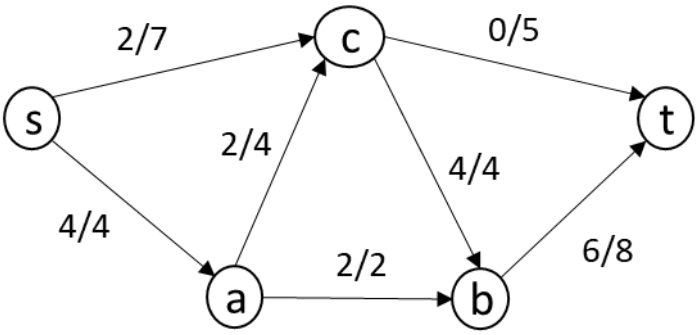
ב ה צ ל ח ה !

♣ שאלה 1 (20 נק')

♣ **1.1** (10 נק') באיור נתונה רשת זרימה עם זרימה חוקית. כל קשת $e \in E$ מסומנת ב-$f(e)/c(e)$.

ידוע שהזרימה התקבלה מביצוע **שתי** איטרציות של אלגוריתם Ford & Fulkerson.

אין צורך לנמק את תשובותיכם לסעיפים הבאים.



א. מה היה מסלול השיפור באיטרציה הראשונה?

s-a-c-b-t

ב. מה היה מסלול השיפור באיטרציה השנייה?

s-c-a-b-t

ג. הציעו מסלול שיפור לאיטרציה הבאה (השלישית):

s-c-t

ד. מה ערך זרימת המקסימום ברשת?

11

ה. כתבו את כל קבוצות הצמתים המגדירות חתך-מינימום ברשת:

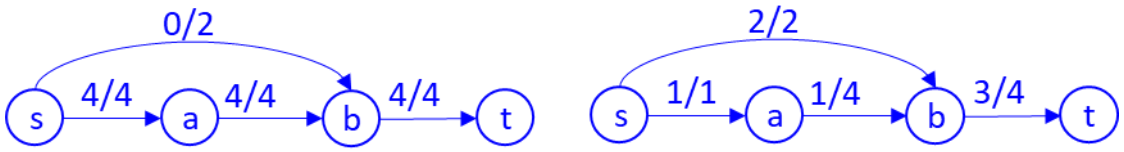
{s}, {s,a,c}

1.2 (10=2*5 נק') תהי G=(V,E) רשת זרימה. נתונה קשת $e_1 \in E$, וידוע שאם מקטינים את הקיבול של e_1 ב-3, אז ערך זרימת המקסימום ברשת קטן ב-1.

הוכיחו או הפריכו כל אחת מהטענות הבאות (בנפרד):

♣ א. לפני שינוי הקיבול, בכל זרימת מקסימום, $f(e_1) < c(e_1)$.

False. Let e_1=(s,a). In the left network, the max-flow value is 4, and e_1 is saturated.
In the right network, the capacity of e_1 reduces by 3, and the max-flow value is 3.



♣ ב. אחרי שינוי הקיבול, בכל זרימת מקסימום, $f(e_1) = c(e_1)$.

True. Since the value of the max-flow reduces, it must be that e_1 is in every min-cut of the new network, and therefore, must be saturated.

Note: An edge is downward critical iff reducing its capacity by any value (even by ε) results in reduced max-flow. The "even by ε" is important. For example, the edge (sa) in the figure above is not downward critical – if its capacity is reduced by x≤2, then the max-flow is not hurt. Therefore, part b cannot be justified 'since e_1 is downward critical'.



♣ שאלה 2 (14 נק')

n ילדות מגיעות לאולם שבו נערכת הופעה, ומעוניינות לקנות כרטיסים. הילדה ה-i מעוניינת לקנות t_i כרטיסים. מחיר הכרטיסים עולה ככל שעובר הזמן. מחיר הבסיס לכרטיס אחד הוא 10 ₪. הילדות צריכות לעמוד בתור, וידוע שהילדה ה-k בתור תשלם עבור כל כרטיס (10+k) ₪ (בלי תלות בכמות הכרטיסים שקונה).
אם למשל $t_1=5, t_2=3, t_3=6, n=3$ והן יעמדו בתור על פי סדר האינדקסים אז המחיר הכולל שישלמו יהיה $6 \cdot (10+3) + 3 \cdot (10+2) + 5 \cdot (10+1)$.

הציעו **אלגוריתם חמדן** הקובע את הסדר שבו כדאי לילדות לעמוד בתור כך שהמחיר הכולל שישלמו יחד יהיה נמוך ככל שניתן.

1. (4 נק') תארו את האלגוריתם, וציינו מה סיבוכיות הזמן שלו.
2. (10 נק') ציינו והוכיחו את ה-greedy choice property.

.

Algorithm: Sort the girls such that $t_1 \geq t_2 \geq .. \geq t_n$.
The order in the line is by the sorting.

Time complexity: O(n log n).

GCP: There exists an optimal order in which girl #1 (alternatively, a girl with maximal t_i) is first.

Proof. Let A be any order, and assume that the first girl is girl $a \neq 1$, and that girl #1 is in the k-th position.
By switching the positions of these two girls, the payments of the other girls do not change.
The total payments of girl 1 and girl a before the swap is $t_a(10+1)+t_1(10+k) = 10t_a + 10t_1 + t_a+kt_1$.
The total payments of girl 1 and girl a after the swap is $t_a(10+k)+t_1(10+1) = 10t_a + 10t_1 + kt_a+t_1$.

Since $k>1$ and, by the sorting, $t_1 \geq t_a$, we have that $(k-1)t_a \leq (k-1)t_1$. Therefore, $kt_a+t_1 \leq t_a+kt_1$, implying that the swap can only reduce the total payments. We conclude that A can be modified to agree with the algorithm without hurting the total payment.



שאלה 3 (18 נק')

בוב המיניון מגיע לשביל המורכב מ-n מרצפות. לכל $1 \le i \le n$,במרצפת ה-i מונח אשכול בננות שיש בו b_i בננות. ניתן להניח שכל ערכי b_i הם מספרים זוגיים גדולים מ-16.
בוב יכול לאסוף בננות מהאשכולות על פי הסדר שלהם בשביל בהתאם לכללים הבאים:
בכל אשכול $1 \le i \le n$ בוב יכול לקחת את כל הבננות ולפרוש (לצאת מהשביל), או לקחת מחצית מהבננות באשכול ולדלג על המרצפת הבאה (לא לקחת כלום מהאשכול ה-i+1), או לקחת שמונה בננות ולעבור למרצפת הבאה.
אם לדוגמה n=7 והאשכולות הם בגדלים (משמאל לימין) 24,26,60,70,32,40,26, אז בוב יכול לאסוף 26+8+8+8+8+8+8 או 40+0+35+0+13+8 או 70+8+0+12בננות (יתכנו סדרות נוספות).

הציעו אלגוריתם המבוסס על **תכנות דינאמי** המחשב מהו מספר הבננות המקסימלי שבוב יכול לאסוף.
לכל $1 \le i \le n$ נסמן ב- $P_{all}[i]$ את מספר הבננות המקסימלי שבוב יכול לאסוף מ-i המרצפות הראשונות, בהנחה שלוקח את כל הבננות באשכול ה-i. ב-$P_{half}[i]$ את מספר הבננות המקסימלי שבוב יכול לאסוף מ-i המרצפות הראשונות, בהנחה שלוקח מחצית מהבננות באשכול ה-i, וב-$P_8[i]$ את מספר הבננות המקסימלי שבוב יכול לאסוף מ-i המרצפות הראשונות, בהנחה שלוקח 8 בננות מהאשכול ה-i.

3.1 (2 נק') מהו הביטוי המתאר את ערך הפתרון (אין צורך לנמק)?

$MAX_{1 \le k \le n} \; P_{all}[k]$

♣ **3.2** (12 נק') כתבו נוסחת תכנות דינאמי לחישוב P_{all}, P_8,P_{half}. הפרידו בין הבסיס למקרה הכללי. הסבירו את הנוסחה במילים.

Base case: for i=0, $P_8[0]=P_{half}[0]=0$.

for i=1, $P_{all}[1]=b_1$, $P_8[1]=8$ $P_{half}[1]=b_1/2$

General case for i>1
$P_{all}[i]= b_i + \max(P_8[i-1], P_{half}[i-2])$
$P_8[i]= 8 + \max(P_8[i-1], P_{half}[i-2])$
$P_{half}[i]= b_i/2 + \max(P_8[i-1], P_{half}[i-2])$

(it is possible not to define i=0 in the base case, and have a different formula for i=2. Specifically, $P_{all}[2]= b_2 + 8$, $P_8[2]= 16$, $P_{half}[2]= b_2/2 + 8$. It is also possible to define $P_{half}[-1]=0$, the entries of i=0 as above, and start the general case from i=1).

Explanation: In every tile, we take the amount of bananas that fit the table description, and add the maximal number of bananas that could be collected from previous tiles, distinguishing between the two options to reach tile i.

♣ **3.3** (4 נק') תארו את טבלת התכנות הדינאמי, את גודלה ואת סדר מילוי הערכים בטבלה. נתחו את סיבוכיות הזמן של האלגוריתם כפונק' של n. נמקו את תשובתכם.

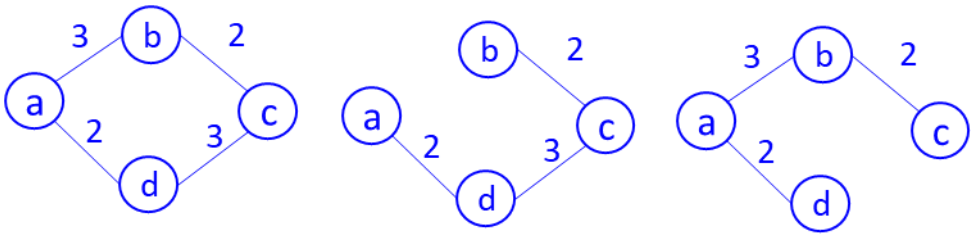
The algorithm fills three 1-dim tables of size n (or a single n*3 table).
For all i, all three entries of index i (in the 3 tables) are filled before moving to index i+1.
Each entry requires $\Theta(1)$ time, therefore the total time complexity is $\Theta(n)$.
Computing the maximal entry in P_{all}, requires additional $\Theta(n)$ [no points were reduced for ignoring this].

♣ שאלה 4 (18=6*3 נק')

יהי $G = (V, E)$ גרף פשוט, לא מכוון, עם מחירים חיוביים על הקשתות. נתון **עץ פורש מינימום** T של G, וכן נתונים שני צמתים $a, b \in V$. יהי p המסלול ב-T שמחבר בין a ל-b.

♣ **4.1** נסמן ב-$mincost(p)$ את המחיר של הקשת הזולה ביותר ב-p.
הוכיחו או הפריכו: בכל עפ"מ של G, מחיר הקשת הזולה ביותר במסלול המחבר בין a ל-b הוא $mincost(p)$.

Counter example: For the graph G in the left figure, and T=middle figure, p=a-d-c-b, and $mincost(p) = 2$. The graph has another MST (right figure) in which the cheapest edge in the path connecting a and b is 3.



♣ **4.2** נסמן ב-$maxcost(p)$ את המחיר של הקשת היקרה ביותר ב-p.
הוכיחו או הפריכו: בכל עפ"מ של G, מחיר הקשת היקרה ביותר במסלול המחבר בין a ל-b הוא $maxcost(p)$.

Proof: Assume by contradiction that there exists an MST T' in which a and b are connected by a path p', such that $maxcost(p) \neq maxcost(p')$.
Assume w.l.o.g., that $maxcost(p) < maxcost(p')$.
The definition of $maxcost(p)$ implies there exists a path in G connecting a and b consisting of edges of cost at most $maxcost(p)$.
By the claim proven in class, in every MST, a and b are connected by a path in which all edges have cost at most $maxcost(p)$, contradicting the fact that $maxcost(p')$ is higher.

♣ **4.3** נתון שהעפ"מ T מתקבל מריצה של אלג' $Prim$ המתחילה בצומת a.
נסמן $p = a - \cdots - u - b$, כלומר, u הוא הצומת האחרון לפני b ב-p (יתכן $u = a$).
הוכיחו או הפריכו: הקשת (u, b) היא האחרונה מבין קשתות p שמצטרפת לעפ"מ T.

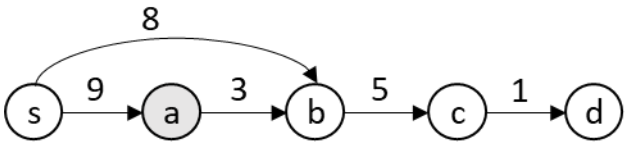
Proof: In a run of Prim, the edges added to T form a single connected component. Since the run begins at 'a', it must be that 'a' belongs to this component. If the last edge in p that is added to T is not (u,b), then 'b' is already in the connected component, that is, there is already a path between a and b in T before the last edge in p is added. Therefore, the addition of the last edge closes a cycle. A contradiction to the fact that Prim returns a tree.

♣ **<u>שאלה 5 (18 נק')</u>**

נתון גרף מכוון $G = (V, E)$, צומת $s \in V$, וצומת $a \in V$. תהי $c: E \to \mathbb{R}^+$ פונקציית מחיר חיובית על הקשתות. הוחלט על מדיניות התשלומים הבאה: אם מסלול עובר בצומת a, אז מחיר המעבר בקשת שיוצאת מ-a הוא 1, מחיר המעבר בקשת שאחריה במסלול הוא 2, וכך הלאה (מחיר המעבר בקשת ה-k אחרי שעוברים בצומת a הוא k). עד המעבר ב-a או במסלול שלא עובר כלל ב-a, המחירים הם על פי פונקציית המחיר c הנתונה.

נסמן ב-$cost(s, v)$ את מחיר המסלול הזול ביותר מ-s ל-v על פי מדיניות התשלומים הזו. הציעו אלגוריתם יעיל המוצא לכל צומת $v \in V$ את $cost(s, v)$.
תארו את האלגוריתם במילים ובאופן פורמלי, נמקו את נכונותו (כתבו את הטענה המרכזית והצדיקו אותה בקצרה), ונתחו את סיבוכיות הזמן שלו.

דוגמה: בגרף שבאיור, מחיר המסלול s-a-b-c הוא 9+1+2=12, מחיר המסלול s-b-c הוא 8+5=13, לכן cost(s,c)=12. מחיר המסלול s-a-b-c-d הוא 9+1+2+3=15, מחיר המסלול s-b-c-d הוא 8+5+1=14, לכן cost(s,d)=14.



נא לענות בדף הבא.

פתרון שאלה 5

Algorithm:

1. Let G' be the graph produced from G by removing all the edges in $out(a)$.
2. Run Dijkstra on G' with s as a source. Denote the output by $d'(s,v)$.
3. Run BFS of G, with a as a source. Denote by $label(a,v)$ the output of this run.
4. For every node v∈V, cost(s,v)=minimum($d'(s,v)$, $d'(s,a) + \frac{label(a,v)(label(a,v)+1)}{2}$).

In steps 1 and 2 we compute for every v∈V the length of a shortest path from s to v that does not pass in a (or only enters a for v= a). In step 3, we compute for every v∈V the minimal number of edges in a path from a to v. In step 4, we return for every node the minimum cost between two options: a path that does not pass in a, and a path that passes in a.

Correctness:

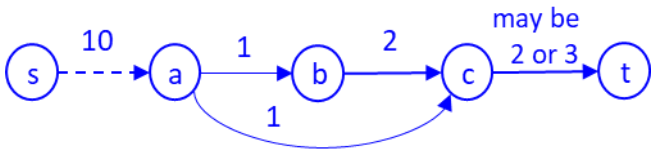
There is a path from s to v with cost x in G ff there is path from s to v in G' that either
(1) is of length x and does not go through a
(2) is comprised of a path of length x-(1+2+...+k) to a, and a path with k edges from a to v.

In particular, if the cheapest path passes in a, then its cost is d'(s,a)+ (1+2+..+label(a,v)), and if the cheapest path does not pass in a then its cost is d'(s,v).

Time complexity:
Step 1: O(|V|+|E|). G' has |V| nodes and at most |E| edges.
Step 2: O(|E|+|V|log|V|) assuming implementation of Dijkstra with Fib. heaps.
Step 3: O(|V|+|E|)
Step 4: O(|V|)

All together: O(|E|+|V|log|V|)

Common mistake: Run BFS from a, make two copies of the graph where the second copy is reached by passing in a. In the second copy, the weight of (u,v) is label(v) in BFS(a). Then run Dijkstra(s_0). The problem with this solution is that there may be several ways to reach a node, each of them by a path with a different number of edges. For example, in this graph, the edge (ct) may be second or third in a path that leaves a). The path s-a-b-c-t has cost 10+1+2+3 while in your construction, since label(t)=2, it has cost 10+1+2+2.



There were several variants of this mistake. In most of them, the shortest path is computed correctly, so no points were reduced in the 'algorithm' part. However, most of the students who used this approach wrote a wrong statement in the 'correctness' part (and lost 3-4 points, depending on the formulation of the claim). For example, it is not true that there is a path with 3 edges after a, iff the graph you built has a path whose cost is d(s,a)+(1+2+3).

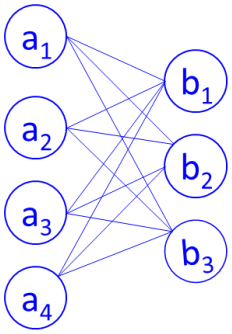
♣ שאלה 6 (12=6*2 נק')

בכל אחד משני הסעיפים הבאים עליכם לתאר גרף **דו-צדדי, פשוט, קשיר, ולא מכוון**, $G = (A \cup B, E)$ המקיים תכונה מסוימת, ושיש בו **מספר קשתות מקסימלי**.
נסמן ב- $a_1, a_2, \dots, a_{|A|}$ ו- $b_1, b_2, \dots, b_{|B|}$ את הצמתים ב- A וב- B.

- כתבו מהו מספר הקשתות בגרף כפונקציה של $|A|$ ו- $|B|$.
- נמקו מדוע זהו המספר המקסימלי של קשתות בגרף דו-צדדי פשוט וקשיר המקיים את התכונה.
- תארו באיור גרף מתאים (ציירו גרף שבו $|A| = 4, |B| = 3$).

♣ **6.1 בריצת $\mathbf{BFS}(a_1)$** מתקבל עץ פורש מכוון שבו $a_2, \dots, a_{|A|}$ הם עלים (צמתים בעלי out-degree=0). ייתכנו עלים נוספים בעץ.

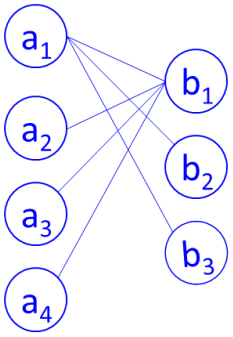
The number of edges is |A|·|B|. This is the maximal number since G is simple. All the nodes in $a_2, \dots, a_{|A|}$ are leaves, since they are all labeled '2' and all the nodes in B are labeled '1'.



♣ **6.2** בריצת $\mathbf{DFS}(a_1)$ מתקבל עץ פורש מכוון שבו $a_2, \dots, a_{|A|}$ הם עלים (צמתים בעלי out-degree=0). ייתכנו עלים נוספים בעץ.

The number of edges is |A|+|B|-1. This is the maximal number since G must be a tree.
Assume by contradiction that G includes a cycle, and let a_k be the first among $a_2, \dots, a_{|A|}$ that belongs to a cycle and is marked by DFS. When a_k is marked, it has at least one unmarked neighbor in B (by the cycle, it has at least two neighbors), contradicting the property that it is a leaf in the DFS tree.

(or any other tree)



Common mistake: it is not sufficient to explain why a specific graph, that you provide has a maximal number of edges. Your justification should cover any graph with |A|+|B|-1 edges.